

动态本地可搜索对称加密 (Dynamic Local Searchable Symmetric Encryption)

目录

- 1. 引言
- 2. 技术概述
- 3. 预备知识
- 4. 分层两选分配 (Layered Two-Choice Allocation)
- 5. 动态分页高效 SSE (LayeredSSE)
- 6. 泛用局部转换 (Generic Local Transform)
- 附录

1. 引言

- 提出了内存高效的动态可搜索加密方案，解决传统静态方案的局限性。
- 核心目标：
 - 存储效率 (Storage Efficiency) : $O(1)$
 - 本地性 (Locality) : $O(1)$
 - 读取效率 (Read Efficiency) : 接近理论下限。

2. 技术概述

- 主要创新：
 - 层次化两选分配算法 (Layered 2-Choice Allocation, L2C)
 - 泛用局部转换 (Generic Local Transform)

3. 预备知识

- 可搜索对称加密 (SSE):
 - 客户端可在加密数据中搜索特定关键词，同时服务器尽量不泄露查询数据。
 - 性能指标：
 - 本地性 (Locality)：数据存储的分散程度。
 - 分页效率 (Page Efficiency)：查询所需的内存页数。
-

算法 1 分层两选分配 (Layered 2-Choice Allocation)

L2C.Setup($n, \{b_1, w_1\}, \dots, \{b_n, w_n\}$) 

功能：

构建分层两选分配数据结构，将元素（球）根据权重分配到适当的桶中。

步骤解析：

1. 初始化参数：

- 计算超级桶的数量：

$$m = \left\lceil \frac{w_{max}}{\delta(\lambda) \log \log w_{max}} \right\rceil$$

其中 w_{max} 是最大权重， $\delta(\lambda)$ 是依赖安全参数的函数。

- 初始化 m 个空超级桶集合： $B_1, \dots, B_m$ 。

2. 分配元素到超级桶：

- 遍历输入集合 $\{b_i, w_i\}$ ：
 - 使用哈希函数生成两个候选桶的索引： $\alpha_1, \alpha_2 \leftarrow H(b_i)$,

- 调用 `L2C.InsertBall(b_i, w_i, B_{\alpha_1}, B_{\alpha_2})`，为当前球选择合适的桶并插入。

3. 返回结果：

- 返回分配完成后的超级桶集合 $B_1, \dots, B_m$ 。

L2C.InsertBall(b, w, B₁, B₂)

功能：

将球 b 根据其权重 w 插入到更适合的桶中，确保负载均衡。

步骤解析：

1. 划分权重区间：

- 将权重划分为 $\log \log m$ 个区间，每个区间的范围为：

$$[0, 1/\log m]_{\mathbb{R}}, (1/\log m, 2/\log m]_{\mathbb{R}}, \dots, (2^{\log \log m - 1}/\log m, 1]_{\mathbb{R}}$$

2. 确定目标桶：

- 根据球的权重 w 所属的区间 k ：
 - 若 $k = 1$ ，直接选择桶 B_{α_1} 。
 - 若 $k > 1$ ，比较 B_{α_1} 和 B_{α_2} 当前球的数量，将 b 插入球更少的桶。

3. 完成插入：

- 将球 b 添加到目标桶中，更新桶的状态。

L2C.UpdateBall(b_{old}, w_{old}, w_{new}, B₁, B₂)

功能：

更新球的权重 w ，并在必要时将球重新分配到适当的桶中。

步骤解析：

1. 检查权重变化：

- 如果 w_{old} 和 w_{new} 属于同一区间，直接在当前桶中更新权重即可。
- 如果 w_{old} 和 w_{new} 跨越了不同区间，则需要重新分配：
 - 将旧球 b_{old} 标记为残留球并移出原桶。
 - 调用 `L2C.InsertBall(b_{old}, w_{new}, B_1, B_2)` 重新插入球到合适的桶。

总结说明：

- `L2C.Setup` 是核心，负责初始化超级桶结构并分配元素；
- `L2C.InsertBall` 实现具体的分配逻辑，确保负载均衡；
- `L2C.UpdateBall` 动态调整球的权重，支持实时更新分配策略。

算法总结

-  **作用：** 实现权重分布的动态平衡，提升存储系统的负载均衡能力。
-  **亮点：**
 - 分层逻辑巧妙划分权重范围。
 - 动态更新机制支持权重变化。

算法 2 动态分页高效 SSE (LayeredSSE)

核心功能

- 在 L2C 基础上构建的动态 SSE 系统。
- 特点：**

- 支持更新和查询操作。
- 利用分页分配技术实现存储高效性。

LayeredSSE.KeyGen(1^λ)

功能：

生成加密密钥，用于数据库的安全操作。

步骤解析：

1. 生成加密密钥：

- 根据安全参数 λ ，生成对称加密密钥 K_{Enc} 。

2. 返回密钥：

- 输出加密密钥 $K = K_{Enc}$ 。

LayeredSSE.Setup(K, N, DB)

功能：

初始化系统，建立加密的索引结构并存储加密数据库。

步骤解析：

1. 初始化超级桶：

- 调用 `L2C.Setup` 分配超级桶： $\{B_1, \dots, B_m\} \leftarrow L2C.Setup(\{w_i, DB(w_i)\}_{i=1}^W, N/p)$ 其中 N 是数据库大小， p 是分组大小。

2. 填充超级桶：

- 使用 0 填充每个桶的容量为： $p \cdot c \log \log(\lambda) \log \log(N/p)$

3. 加密超级桶内容：

- 对每个桶 B_i ，加密内容： $B_i^{Enc} \leftarrow \text{Enc}_K(B_i)$

4. 返回结果：

- 输出加密数据库 $EDB = \{B_1^{Enc}, \dots, B_m^{Enc}\}$ 。

LayeredSSE.Search(K, w, EDB)

功能：

对给定关键词 w 进行搜索，获取对应的文档集合。

步骤解析：

1. 客户端：

- 返回关键词 w 。

2. 服务器：

- 计算两个候选桶索引： $\alpha_1, \alpha_2 \leftarrow H(w)$
- 返回加密桶内容 $B_{\alpha_1}^{Enc}, B_{\alpha_2}^{Enc}$ 。

3. 客户端：

- 解密桶内容，提取相关文档集合。

LayeredSSE.Update(K, w, L', add, EDB)

功能：

对关键词 w 的索引进行动态更新，添加新文档。

步骤解析：

1. 客户端：

- 返回关键词 w 。

2. 服务器：

- 计算桶索引： $\alpha_1, \alpha_2 \leftarrow H(w)$
- 返回对应桶的加密内容。

3. 客户端：

- 解密桶内容，更新文档集合。
- 加密更新后的内容，返回给服务器。

4. 服务器：

- 替换旧的加密桶内容为更新后的数据。

总结说明：

1. LayeredSSE.KeyGen 生成加密密钥；
 2. LayeredSSE.Setup 初始化加密数据库，建立索引结构；
 3. LayeredSSE.Search 实现高效的关键词检索；
 4. LayeredSSE.Update 动态更新关键词索引，支持系统的灵活性和扩展性。
-

泛用局部转换 (Generic Local Transform)



算法 3 1C-Alloc

1C-Alloc.Fetch

功能：

从输入 m, w, ℓ 中返回与关键词 w 相关的 **唯一 ℓ -超级桶集合**。
这是用于快速定位存储数据的一部分。

步骤解析：

1. 输入参数：

- m ：总桶数量。
- w ：关键词。
- ℓ ：当前的超级桶层级。

2. 算法步骤:

- Step 1: 计算 $\ell' = 2 \lfloor \log \ell \rfloor$.
 - 这个计算用于确定最接近 ℓ 的 2 的幂, 作为分配规则基础。
 - Step 2: 检查 ℓ' 是否大于或等于 m' .
 - 如果 $\ell' \geq m$:
 - 返回 $\{0, \dots, m - 1\}$, 即返回所有桶。
 - 如果 $\ell' < m$:
 - Step 3: 计算超级桶编号 $i = \lfloor H(w)/\ell' \rfloor$.
 - $H(w)$ 是通过哈希函数计算关键词 w 的哈希值。
 - 用 ℓ' 将哈希值分区, 得到超级桶编号。
 - Step 4: 返回超级桶的编号集合 $\ell' * i, \dots, \ell' * i + \ell' - 1$ 。
-

1C-Alloc.Add +

功能:

将新的关键词 w 添加到正确的桶中, 保证它在对应的 ℓ -超级桶范围内。

步骤解析:

1. 输入参数:

- m : 总桶数量。
- w : 关键词。
- ℓ : 当前的超级桶层级。

2. 算法步骤:

- Step 1: 计算 $\ell' = \ell \bmod m$.
 - 将当前层级 ℓ 映射到总桶数量范围。
- Step 2: 计算 $\ell' = 2 \lfloor \log(\ell + 1) \rfloor$.

- 取最近的 2 的幂以简化分配。
 - Step 3: 确定超级桶编号 $i = \lfloor H(w)/\ell' \rfloor$ 。
 - Step 4: 根据 $2H(w)/\ell'$ 的奇偶性选择目标桶:
 - 如果是偶数: 返回桶 $\ell' * i + \ell$ 。
 - 如果是奇数: 返回桶 $\ell' * i + \ell - \ell'/2$ 。
-

总体说明:

- `1C-Alloc.Fetch` 用于确定哪些桶与关键词关联, 这是一种定位策略。
 - `1C-Alloc.Add` 通过特定分配规则将关键词插入对应的超级桶, 保证动态调整时仍能维持高效性和一致性。
 - 将分页高效方案转化为具有良好本地性的方案。
 - 步骤:
 - 使用“溢出方案”处理部分数据。
 - 结合“填充和分割”技术优化存储。
-

算法4 ClipOSSE

ClipOSSE.KeyGen

功能:

生成密钥, 用于数据加密和伪随机函数 (PRF) 的计算。

步骤解析:

1. 生成密钥 K 和 K_{PRF} :
 - K : 用于数据加密。
 - K_{PRF} : 用于伪随机函数 (PRF) 的计算。
 2. 返回密钥对 (K, K_{PRF}) 。
-

ClipOSSE.Setup(N, DB)

功能:

初始化 ClipOSSE 系统, 包括建立索引和加密数据库。

步骤解析:

1. 初始化参数:

- 设定 $m = 2\lceil \log(N / \log \log N) \rceil$ 和 $\tau = \lceil \alpha \log \log N \rceil$, 其中 N 为数据库大小, α 是常量。
- 初始化桶集合 $B_0, \dots, B_{m-1}$, 加密数据库 E_{DB} 和裁剪集合 $clip$ 。

2. 处理数据库 DB 中的每个关键词:

- 为关键词 w 生成伪随机密钥 $K_w = PRF_{K_{PRF}}(w)$ 。
- 加密层级信息 $T[w] = Enc_{K_w}(\ell)$ 。
- 对每个 $t = 1$ 到 ℓ :
 - 使用 $1C\text{-Alloc.Add}$ 分配超级桶。
 - 如果桶大小小于阈值 τ , 将元素添加到桶中。
 - 如果桶满, 将多余元素转移到裁剪集合 c 。
- 如果裁剪集合非空, 将其加入 $clip$ 。

3. 生成加密数据库:

- 对每个桶内容进行加密, 存入 $B_{Enc}[i]$ 。
- 返回加密数据库 $(T, (B_{Enc}[i]), clip)$ 。

ClipOSSE.Search(w)

功能:

执行关键词 w 的检索。

步骤解析:

1. 客户端:

- 使用伪随机函数计算 $K_w = PRF_{K_{PRF}}(w)$, 并发送给服务器。

2. 服务器：

- 解密 $\ell = Dec_{K_w}(T[w])$ 。
- 使用 `1C-Alloc.Fetch` 获取与关键词相关的超级桶集合 `s`。
- 返回桶中加密的内容 $B_{Enc}[i]$ 。

3. 客户端：

- 根据返回的加密数据进一步处理结果。

ClipOSSE.Update(w, e)

功能：

更新关键词 `w` 的索引，将新元素 `e` 添加到系统中。

步骤解析：

1. 客户端：

- 计算 $K_w = PRF_{K_{PRF}}(w)$ ，并发送给服务器。

2. 服务器：

- 解密层级信息 $\ell = Dec_{K_w}(T[w])$ 。
- 使用 `1C-Alloc.Add` 确定目标超级桶。
- 返回对应的加密桶 $B_{Enc}[i]$ 。

3. 客户端：

- 解密桶内容 $B = Dec_K(B_{Enc}[i])$ 。
- 检查桶容量：
 - 如果容量小于 τ ，将元素添加到桶中。
 - 如果容量已满，将新元素转移到裁剪集合 `clip`。
- 将更新后的桶重新加密并发送回服务器。

4. 服务器：

- 更新加密桶内容 $B_{Enc}[i]$ 。

总结说明：

- KeyGen 负责生成系统的核心密钥；
- Setup 初始化数据库索引并加密存储；
- Search 实现关键词的快速检索；
- Update 支持高效的动态更新操作，同时利用裁剪机制（clip）处理桶满的情况。

ClipOSSE 算法的设计目标是提升动态可搜索加密的效率，同时保证安全性和性能平衡。

算法 5 Generic Local Transform (Local[PE-SSE])

Local[PE-SSE].KeyGen

功能：

生成密钥，用于加密数据及伪随机函数计算。

步骤解析：

1. 生成密钥：

- K ：用于数据加密。
- K_{PRF} ：用于伪随机函数（PRF）。

2. 返回密钥对：

- 输出 (K, K_{PRF}) 。

Local[PE-SSE].Setup(DB, N)

功能：

初始化分层加密搜索数据库。

步骤解析：

1. 初始化参数：

- 确定分层数量 $n_{level} = \lceil N / \log^d N \rceil$ 。
- 初始化加密数据库 EDB 和每个关键词的层级结构 PE-SSE_i。

2. 为每个关键词构建索引：

- 遍历数据库 DB：
 - 使用伪随机函数 $K_w = \text{PRF}_{K_{\text{PRF}}}(w)$ 生成关键词密钥。
 - 加密关键词计数 $T[w] = \text{Enc}_{K_w}(|S|)$ 。
 - 按层级 $i = 1$ 到 n_{level} 将关键词加入对应层。

3. 生成最终加密数据库：

- 返回 EDB 和索引结构 $T[w]$ 。

Local[PE-SSE].Search(w)

功能：

检索关键词 w 对应的文档集合。

步骤解析：

1. 客户端：

- 通过 $K_w = \text{PRF}_{K_{\text{PRF}}}(w)$ 提取关键词密钥，并发送给服务器。

2. 服务器：

- 解密 $\ell = \text{Dec}_{K_w}(T[w])$ ，确定关键词对应的层索引。
- 在对应的层级中执行搜索，提取相关文档。

3. 客户端：

- 获取搜索结果，解密文档集合并返回给用户。

Local[PE-SSE].Update(w, e)

功能：

更新关键词 w ，将新文档 e 添加到数据库中。

步骤解析：

1. 客户端：

- 计算关键词密钥 $K_w = \text{PRF}_{K_{\text{PRF}}}(w)$ ，发送到服务器。

2. 服务器：

- 解密关键词层级 $\ell = \text{Dec}_{K_w}(T[w])$ 。
- 确定更新层级，并返回对应加密层的内容。

3. 客户端：

- 解密层内容，将新文档 e 添加到关键词对应的集合中。
- 将更新后的集合重新加密，返回给服务器。

4. 服务器：

- 更新加密数据库的层级内容。

3 算法总结

-  **作用：** 实现高效的分层搜索加密，支持动态更新。
-  **亮点：**
 - i. 支持增量更新，无需重新构建数据库。
 - ii. 通过分层优化搜索效率。

总结

- **创新点：** 首次实现动态内存高效 SSE。
- **应用前景：**
 - 加密云存储、消息服务。
 - 提升现有搜索加密系统性能。

附录

1. 这篇论文中提到的五个主要算法分别解决了哪些特定问题？它们之间是如何协作的？

- L2C：通过分层机制实现动态数据的高效检索。
 - LayeredSSE：改进 L2C，将关键词数据划分为多层，减少查询范围。
 - 1C-Alloc：实现动态关键词桶分配，解决了索引动态更新的负载均衡问题。
 - ClipSSE：通过裁剪机制处理桶超载，确保高效动态更新和检索。
 - Generic Local Transform：整合上述算法为通用框架，适应不同场景。
- 这些算法协作构建了一个支持动态、局部化存储和高效查询的加密体系。

2. 在 LayeredSSE 中，分层设计是如何影响关键词的索引和检索过程的？每一层的作用和组织方式是什么？

分层设计通过将数据按层级划分，每层维护一个较小的数据范围，查询时从低层到高层依次搜索，提高检索效率。低层负责存储频繁访问的关键词，高层存储稀疏数据，分层机制减少了查询的范围和数据量。

3. 1C-Alloc 中，Fetch 函数具体是如何根据关键词定位到超级桶的？它使用的哈希计算有何特殊之处？

Fetch 函数通过计算关键词的哈希值 $H(w)$ 并除以层级大小 ℓ' 来定位超级桶集合。它的特殊之处在于通过分层哈希定位，既能快速检索，又能适应动态数据更新，避免重新分配。

4. 1C-Alloc 中，Add 函数的执行逻辑是怎样的？它是如何处理桶容量不足的情况？

Add 函数根据关键词的哈希值和当前层级 t 动态分配数据到超级桶。

1. 如果数据项的数量 l 超过了桶的总数 m ，则 Fetch 返回所有桶，而 Add 会选择已经使用过的桶，但会选择它们的 $l \bmod m$ 位置。
2. 这样做确保了即使超出了桶的数量，桶选择也能保持均匀分布，避免某个桶过度拥

挤。

5. ClipOSSE 的 Setup 过程有哪些关键步骤？在构建索引时，如何确保分配的桶不会过载？裁剪集合（Clip）在这个过程中起了什么作用？

初始化分层索引和加密桶；

使用 1C-Alloc 动态分配关键词到桶中；

当桶超载时，将多余数据放入裁剪集合。

裁剪集合缓解了桶的存储压力，确保了数据分布的均衡性。

6. ClipOSSE 的 Search 过程如何结合动态分配算法快速找到相关数据？在查询结果中，如何保证数据的完整性和准确性？

Search 解密查询关键词的层级信息，调用 1C-Alloc 的 **Fetch** 定位相关桶，并从桶中提取加密数据。通过分层设计和桶哈希分配，保证查询范围的局限性，同时维护数据的完整性和正确性。

7. ClipOSSE 的 Update 操作中，当新增的元素导致桶容量不足时，裁剪机制如何介入解决问题？具体是如何转移和存储超额数据的？

当新增数据导致桶容量超过阈值时，多余数据转移到裁剪集合。裁剪集合按层级维护剩余数据，既避免丢失，又确保桶索引高效，同时优化了更新性能。

8. Generic Local Transform 是如何将其他算法模块化组合成一个通用的解决方案的？它的设计有哪些关键步骤？

Generic Local Transform 利用分层存储和裁剪机制统一管理动态更新和检索过程。关键步骤包括：

1. 初始化通用分层模型；
2. 集成 ClipOSSE 的分配和裁剪策略；
3. 动态调整桶和索引，适配不同查询场景。

9. L2C 算法中，“层级划分”具体是如何影响检索效率的？当层数增加时，算法的查询复杂度如何变化？

层级划分将检索范围限制在当前层级，降低了搜索空间。随着层数增加，单层的数据减少，但需要查询的层级数量可能增多，查询复杂度从 $O(N)$ 降至 $O(\log N)$ 。

10. 在所有算法中，桶的分配和检索是一个核心操作。不同算法（如 1C-Alloc 和 ClipOSSE）在分配策略上有哪些异同点？这些差异如何影响了性能？

- **相同点：**ClipOSSE底层使用了1C-Alloc，可以说二者都使用哈希分配关键词到桶，支持动态更新。
- **不同点：**1C-Alloc 关注均衡分配，ClipOSSE 增加了裁剪机制处理超载问题。裁剪机制使 ClipOSSE 更适应动态场景，但增加了一定的存储和查询复杂度。